

Contents

1	Template	2	5.1	LCA - Binary Lifting	18
2	Number Theory	2	5.1.1	Normal LCA	18
2.1	Binary Exponentiation	2	5.1.2	Min/Max weight on the path	18
2.2	Modular inverse	2	5.1.3	Union path	19
2.3	Sieve of Eratosthenes	2	5.1.4	Intersection Path	19
3	Data structures	2	5.2	Centroid Decomposition	19
3.1	Segment Tree	2	5.3	Euler Tour Technique	20
3.2	Merge Sort Tree	3	5.4	Tree Matching	20
3.3	Fenwick Tree	3	5.5	Diameter	20
3.4	Sparse Table	4	5.6	Heavy Light Decomposition	21
3.5	Disjoint Set Union	4	6	Math	22
3.6	Lazy SegTree	5	6.1	FFT	22
3.6.1	Min/Max Range / Single Acces Element	5	6.1.1	XOR	22
3.7	Implicit Treap	5	6.2	Theorems	22
4	Graphs	10	7	Strings	22
4.1	DFS	10	7.1	Suffix Array	22
4.2	BFS	10	8	Others	24
4.3	Topological Sort	10	8.1	Binpow	24
4.4	Flood Fill	10	8.2	Matrix exponentiation	24
4.5	MST	10	8.3	Random	25
4.5.1	Kruskal	10			
4.5.2	Prims	11			
4.6	Dijkstra	11			
4.6.1	Implementation	11			
4.6.2	Space-State graph	12			
4.7	Bellman-Ford	12			
4.7.1	Path	12			
4.7.2	Cycle	12			
4.8	Floyd-Warshall	12			
4.9	SCC Kosaraju	13			
4.10	Flows	13			
4.10.1	Dinic	13			
4.11	2-SAT	15			
4.12	Euler Paths and cycles	16			
4.12.1	Directed Graphs	16			
4.12.2	Unirected Graphs	17			
5	Trees	18			

1 Template

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 #define all(v) v.begin(),v.end()
6 #define rall(v) v.rbegin(),v.rend()
7 #define pb push_back
8 #define sz(a) int(a.size())
9 #define F first
10 #define S second
11 typedef long long ll;
12 typedef vector<int> vi;
13 typedef vector<ll> vll;
14 typedef pair<int,int> pii;
15 typedef pair<ll,ll> pll;
16 const int MOD = 1e9+7;
17 const int INF = INT_MAX;
18 const ll INF64 = LLONG_MAX;
19 const long double EPS = 1e-9;
20 const long double PI = acos(-1.0L);
21 void setIO(string p){
22     freopen((p + ".in").c_str(), "r", stdin);
23     freopen((p + ".out").c_str(), "w", stdout);
24 }
25
26 void solve(){
27
28 }
29
30
31 int main(){
32     ios_base::sync_with_stdio(0);
33     cin.tie(0);
34     cout.tie(0);
35     int tc = 1;
36     //cin >> tc;
37     for (int t = 1; t <= tc; t++)solve();
38 }

```

2 Number Theory

2.1 Binary Exponentiation

2.2 Modular inverse

2.3 Sieve of Eratosthenes

3 Data structures

3.1 Segment Tree

```

1 struct Node{
2     ll val = 0;
3 };
4 Node operator+(Node a,Node b){return {a.val+b.val};}
5 class Segment_Tree{
6 public:
7     int n;
8     vector<Node> tree;
9
10    Segment_Tree(int x = 1e5+10){
11        n = x;
12        tree.resize(2*n+1);
13    }
14
15    void update(int p, ll val){
16        for(tree[p+=n] = {val};p>1;p>>=1)
17            tree[p>>1] = tree[p]+tree[p^1];
18    }
19
20    ll query(int l,int r){
21        Node s;
22        for(l+=n,r+=n ; l<r ; l>>=1 , r>>=1){
23            if(l&1)s = s+ tree[l++];
24            if(r&1)s = s+ tree[--r];
25        }
26        return s.val;
27    }
28 };

```

3.2 Merge Sort Tree

```

1 struct Node{
2     vi val;
3
4     int search(int x){
5         int l = 0, r = sz(val)-1;
6         while(l<=r){
7             int m = l+(r-l)/2;
8             if(val[m] < x) l = m+1;
9             else r = m-1;
10        }
11        return sz(val)-1;
12    }
13 };
14
15 Node operator+(Node a,Node b){
16     int i=0,j=0;
17     Node aux;
18     while (i< sz(a.val) && j<sz(b.val)){
19         if (a.val[i]< b.val[j] ) aux.val.pb(a.val[i++]);
20         else aux.val.pb(b.val[j++]);
21     }
22     while (i<sz(a.val)) aux.val.pb(a.val[i++]);
23     while (j<sz(b.val)) aux.val.pb(b.val[j++]);
24     return aux;
25 }
26 //Call segment Tree

```

3.3 Fenwick Tree

```

1 struct Node{
2     ll n = 0;
3     Node operator+(Node b){return {n+b.n};}
4 };
5
6 class Fenwick_Tree{
7 public:
8     vector<Node> tree;
9     int n;
10    Fenwick_Tree(int x){
11        n = x+1;tree.resize(n);
12    }
13
14    ll sum(int r){
15        r++;
16        Node ans = {0};
17        while(r){
18            ans = ans + tree[r];
19            r -= r&-r;
20        }
21        return ans.n;
22    }
23
24    void update(int id, Node val){
25        id++;
26        while(id < n){
27            tree[id] = tree[id]+val;
28            id += id&-id;
29        }
30    }
31 };

```

3.4 Sparse Table

```

1 struct Node{
2     int val = 0;
3     Node operator+(Node b){return {__gcd(val,b.val)};}
4 };
5
6 class SparseTable{
7     public:
8     vector<vector<Node>> sparse;
9
10    SparseTable(int n=2e5+10){
11        sparse.assign(n , vector<Node> (log2(n)+1));
12    }
13
14    void build(const vi &a){
15        for(int i=0;i<sz(a);i++)
16            sparse[i][0] = {a[i]};
17
18        for(int j=1;(1<<j)<=sz(a);j++)
19            for(int i=0;i+(1<<j)-1 <sz(a);i++)
20                sparse[i][j]= sparse[i][j-1]+sparse[i+(1<<(j-1))][j-1];
21    }
22
23    int query(int l, int r){
24        int len = r-l+1;
25        int k = log2(len);
26        return (sparse[l][k]+sparse[r-(1<<k)+1][k]).val;
27    }
28
29 };

```

3.5 Disjoint Set Union

```

1 class DSU{
2     private:
3     vi parent,rank,size;
4     int c;
5     public:
6     int mx = 0;
7     DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
8         iota(all(parent),0);
9     }
10
11    int find(int u){return parent[u] == u?u:parent[u] = find(parent[u]) ;}
12
13    bool same(int u,int v){return find(u)==find(v);}
14
15    int get_size(int u){return size[find(u)];}
16
17    int count(){return c;}
18
19    void merge(int u,int v){
20        u = find(u);
21        v = find(v);
22        if(u!=v){
23            c--;
24            if(rank[u] > rank[v])swap(u,v);
25            parent[u] = v;
26            size[v] += size[u];
27            if(rank[u] == rank[v])rank[v]++;
28            mx = max(mx,size[v]);
29        }
30    }
31 };

```

3.6 Lazy SegTree

3.6.1 Min/Max Range / Single Acces Element

```

1 struct Node{
2     ll val = 0;
3 };
4
5 struct LazyNode{
6     ll val = 0;
7     bool can(){return val!=0;}
8 };
9
10 Node operator+(Node a, LazyNode b){return {a.val+b.val};}
11 Node operator+(Node a, Node b){return {a.val+b.val};}
12 LazyNode operator+(LazyNode a, LazyNode b){return {a.val+b.val};}
13
14 class Segment_Tree{
15 private:
16     int n,h;
17
18     void apply(int p, ll val){
19         tree[p].val += val;
20         if(p < n)d[p].val += val;
21     }
22
23     void build_lazy(int p){
24         while(p>1){
25             p >>= 1;
26             tree[p] = (tree[p*2]+tree[p*2+1])+d[p];
27         }
28     }
29
30     void push(int p){
31         for(int s = h;s>0;--s){
32             int i = p>>s;
33             if(d[i].can()){
34                 apply(2*i,d[i].val);
35                 apply(2*i+1,d[i].val);
36                 d[i].val = 0;
37             }
38         }
39     }

```

```

40
41 public:
42     vector<Node> tree;
43     vector<LazyNode> d;
44     Segment_Tree(int x = 1e5+10){
45         n = x;
46         tree.resize(2*n+1);
47         d.resize(n+1);
48         h = sizeof(int)*8 - __builtin_clz(n);
49     }
50     void build(vi &arr){
51         for(int i=0;i<n;i++)tree[n+i] = {arr[i]};
52         for(int i = n-1; i>0;i--)tree[i] = tree[i*2]+tree[i*2+1];
53     }
54     ll query(int l, int r){
55         l += n;r+=n;
56         push(l);
57         push(r-1);
58         Node s;
59         for(;l<r; l>>=1,r>>=1){
60             if(l&1)s = s+ tree[l++];
61             if(r&1)s = s+ tree[--r];
62         }
63         return s.val;
64     }
65
66     void range_increment(int l,int r,ll val){
67         l += n;
68         r += n;
69         int l0=l,r0=r;
70
71         for(;l<r;l>>=1,r>>=1){
72             if(l&1)apply(l++,val);
73             if(r&1)apply(--r,val);
74         }
75         build_lazy(l0);
76         build_lazy(r0-1);
77     }
78
79 };

```

3.7 Implicit Treap

```

1 struct node {
2     int val, sz, prior, lazy, sum, mx, mn, repl;
3     bool repl_flag, rev;
4     node *l, *r, *par;
5     node() {
6         lazy = 0;
7         rev = 0;
8         sum = 0;
9         val = 0;
10        sz = 0;
11        mx = 0;
12        mn = 0;
13        repl = 0;
14        repl_flag = 0;
15        prior = 0;
16        l = NULL;
17        r = NULL;
18        par = NULL;
19    }
20    node(int _val) {
21        val = _val;
22        sum = _val;
23        mx = _val;
24        mn = _val;
25        repl = 0;
26        repl_flag = 0;
27        rev = 0;
28        lazy = 0;
29        sz = 1;
30        prior = rnd();
31        l = NULL;
32        r = NULL;
33        par = NULL;
34    }
35 };
36
37 struct treap {
38     typedef node* pnode;
39     pnode root;
40     map<int, pnode> position; //positions of all the values
41     //clearing the treap
42     void clear() {
43         root = NULL;

```

```

44         position.clear();
45     }
46
47     treap() {
48         clear();
49     }
50
51     int size(pnode t) {
52         return t ? t->sz : 0;
53     }
54     void update_size(pnode &t) {
55         if(t) t->sz = size(t->l) + size(t->r) + 1;
56     }
57
58     void update_parent(pnode &t) {
59         if(!t) return;
60         if(t->l) t->l->par = t;
61         if(t->r) t->r->par = t;
62     }
63     //add operation
64     void lazy_sum_upd(pnode &t) {
65         if( !t or !t->lazy ) return;
66         t->sum += t->lazy * size(t);
67         t->val += t->lazy;
68         t->mx += t->lazy;
69         t->mn += t->lazy;
70         if( t->l ) t->l->lazy += t->lazy;
71         if( t->r ) t->r->lazy += t->lazy;
72         t->lazy = 0;
73     }
74     //replace update
75     void lazy_repl_upd(pnode &t) {
76         if( !t or !t->repl_flag ) return;
77         t->val = t->mx = t->mn = t->repl;
78         t->sum = t->val * size(t);
79         if( t->l ) {
80             t->l->repl = t->repl;
81             t->l->repl_flag = true;
82         }
83         if( t->r ) {
84             t->r->repl = t->repl;
85             t->r->repl_flag = true;
86         }

```

```

87     t->repl_flag = false;
88     t->repl = 0;
89 }
90 //reverse update
91 void lazy_rev_upd(pnode &t) {
92     if( !t or !t->rev ) return;
93     t->rev = false;
94     swap(t->l, t->r);
95     if( t->l ) t->l->rev ^= true;
96     if( t->r ) t->r->rev ^= true;
97 }
98 //reset the value of current node assuming it now
99 //represents a single element of the array
100 void reset(pnode &t) {
101     if(!t) return;
102     t->sum = t->val;
103     t->mx = t->val;
104     t->mn = t->val;
105 }
106 //combine node l and r to form t by updating corresponding queries
107 void combine(pnode &t, pnode l, pnode r) {
108     if(!l) {
109         t = r;
110         return;
111     }
112     if(!r) {
113         t = l;
114         return;
115     }
116     //Beware!!!Here t can be equal to l or r anytime
117     //i.e. t and (l or r) is representing same node
118     //so operation is needed to be done carefully
119     //e.g. if t and r are same then after t->sum=l->sum+r->sum operation
120     ,
121     //r->sum will be the same as t->sum
122     //so BE CAREFUL
123     t->sum = l->sum + r->sum;
124     t->mx = max(l->mx, r->mx);
125     t->mn = min(l->mn, r->mn);
126 }
127 //perform all operations
128 void operation(pnode &t) {
129     if( !t ) return;

```

```

129     reset(t);
130     lazy_rev_upd(t->l);
131     lazy_rev_upd(t->r);
132     lazy_repl_upd(t->l);
133     lazy_repl_upd(t->r);
134     lazy_sum_upd(t->l);
135     lazy_sum_upd(t->r);
136     combine(t, t->l, t);
137     combine(t, t, t->r);
138 }
139 //split node t in l and r by key k
140 //so first k+1 elements(0,1,2,...k) of the array from node t
141 //will be split in left node and rest will be in right node
142 void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
143     if(t == null) {
144         l = null;
145         r = null;
146         return;
147     }
148     lazy_rev_upd(t);
149     lazy_repl_upd(t);
150     lazy_sum_upd(t);
151     int idx = add + size(t->l);
152     if(t->l) t->l->par = null;
153     if(t->r) t->r->par = null;
154     if(idx <= k)
155         split(t->r, t->r, r, k, idx + 1), l = t;
156     else
157         split(t->l, l, t->l, k, add), r = t;
158
159     update_parent(t);
160     update_size(t);
161     operation(t);
162 }
163 //merge node l with r in t
164 void merge(pnode &t, pnode l, pnode r) {
165     lazy_rev_upd(l);
166     lazy_rev_upd(r);
167     lazy_repl_upd(l);
168     lazy_repl_upd(r);
169     lazy_sum_upd(l);
170     lazy_sum_upd(r);
171     if(!l) {

```

```

172     t = r;
173     return;
174 }
175 if(!r) {
176     t = l;
177     return;
178 }
179
180 if(l->prior > r->prior)
181     merge(l->r, l->r, r), t = l;
182 else
183     merge(r->l, l, r->l), t = r;
184
185 update_parent(t);
186 update_size(t);
187 operation(t);
188 }
189 //insert val in position a[pos]
190 //so all previous values from pos to last will be right shifted
191 void insert(int pos, int val) {
192     if(root == NULL) {
193         pnode to_add = new node(val);
194         root = to_add;
195         position[val] = root;
196         return;
197     }
198
199     pnode l, r, mid;
200     mid = new node(val);
201     position[val] = mid;
202
203     split(root, l, r, pos - 1);
204     merge(l, l, mid);
205     merge(root, l, r);
206
207     /*pnode x = new Node(val),merge(root,root,x);*/
208 }
209 //erase from qL to qR indexes
210 //so all previous indexes from qR+1 to last will be left shifted qR-qL
211 //+1 times
212 void erase(int qL, int qR) {
213     pnode l, r, mid;

```

```

214     split(root, l, r, qL - 1);
215     split(r, mid, r, qR - qL);
216     merge(root, l, r);
217 }
218 //returns answer for corresponding types of query
219 int query(int qL, int qR) {
220     pnode l, r, mid;
221
222     split(root, l, r, qL - 1);
223     split(r, mid, r, qR - qL);
224
225     int answer = mid->sum;
226     merge(r, mid, r);
227     merge(root, l, r);
228
229     return answer;
230 }
231 //add val in all the values from a[qL] to a[qR] positions
232 void update(int qL, int qR, int val) {
233     pnode l, r, mid;
234
235     split(root, l, r, qL - 1);
236     split(r, mid, r, qR - qL);
237     lazy_repl_upd(mid);
238     mid->lazy += val;
239
240     merge(r, mid, r);
241     merge(root, l, r);
242 }
243 //reverse all the values from qL to qR
244 void reverse(int qL, int qR) {
245     pnode l, r, mid;
246
247     split(root, l, r, qL - 1);
248     split(r, mid, r, qR - qL);
249
250     if(mid)mid->rev ^= 1;
251     merge(r, mid, r);
252     merge(root, l, r);
253 }
254 //replace all the values from a[qL] to a[qR] by v
255 void replace(int qL, int qR, int v) {
256     pnode l, r, mid;

```



```

257     split(root, l, r, qL - 1);
258     split(r, mid, r, qR - qL);
259     lazy_sum_upd(mid);
260     mid->repl_flag = 1;
261     mid->repl = v;
262     merge(r, mid, r);
263     merge(root, l, r);
264 }
265 //it will cyclic right shift the array k times
266 //so for k=1, a[qL]=a[qR] and all positions from ql+1 to qR will
267 //have values from previous a[qL] to a[qR-1]
268 //if you make left_shift=1 then it will to the opposite
269 void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
270     if(qL == qR) return;
271     k %= (qR - qL + 1);
272
273     pnode l, r, mid, fh, sh;
274     split(root, l, r, qL - 1);
275     split(r, mid, r, qR - qL);
276
277     if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
278     else split(mid, fh, sh, k - 1);
279     merge(mid, sh, fh);
280
281     merge(r, mid, r);
282     merge(root, l, r);
283 }
284 bool exist;
285 //returns index of node curr
286 int get_pos(pnode curr, pnode son = nullptr) {
287     if(exist == 0) return 0;
288     if(curr == NULL) {
289         exist = 0;
290         return 0;
291     }
292     if(!son) {
293         if(curr == root) return size(curr->l);
294         else return size(curr->l) + get_pos(curr->par, curr);
295     }
296
297     if(curr == root) {
298         if(son == curr->l) return 0;

```

```

300         else return size(curr->l) + 1;
301     }
302
303     if(curr->l == son) return get_pos(curr->par, curr);
304     else return get_pos(curr->par, curr) + size(curr->l) + 1;
305 }
306 //returns index of the value
307 //if the value has multiple positions then it will
308 //return the last index where it was added last time
309 //returns -1 if it doesn't exist in the array
310 int get_pos(int value) {
311     if(position.find(value) == position.end()) return -1;
312     exist = 1;
313     int x = get_pos(position[value]);
314     if(exist == 0) return -1;
315     else return x;
316 }
317 //returns value of index pos
318 int get_val(int pos) {
319     return query(pos, pos);
320 }
321 //returns size of the treap
322 int size() {
323     return size(root);
324 }
325 //inorder traversal to get indexes chronologically
326 void inorder(pnode cur) {
327     if(cur == NULL) return;
328     operation(cur);
329     inorder(cur->l);
330     cout << cur->val << '␣';
331     inorder(cur->r);
332 }
333 //print current array values serially
334 void print_array() {
335     // for(int i=0;i<size();i++) cout<<get_val(i)<<' ';
336     // cout<<nl;
337     inorder(root);
338     cout << nl;
339 }
340 bool find(int val) {
341     if(get_pos(val) == -1) return 0;
342     else return 1;

```

```

343     }
344 };
345
346 treap t;
347 //Beware!!!here treap is 0-indexed
348
349 int main() {
350     int i, j, k, n, m, l, r, q;
351     for(i = 0; i < 10; i++) t.insert(i, i * 10);
352     t.cyclic_shift(4, 5, 1);
353     t.update(2, 5, 1);
354     t.replace(2, 5, 100);
355     t.reverse(2, 9);
356     t.replace(2, 5, 200);
357     cout << t.query(0, 7) << nl;
358     t.cyclic_shift(2, 3, 2, 1);
359     cout << t.get_pos(20) << nl;
360     t.erase(2, 2);
361     cout << t.find(30) << nl;
362     t.print_array();
363     return 0;
364 }

```

4 Graphs

4.1 DFS

```

1 void dfs(int u){
2     vis[u] = 1;
3     for(auto v:adj[u]){
4         if(!vis[v])dfs(v);
5     }
6 }

```

4.2 BFS

```

1 queue<int> q;
2 q.push(0);
3 while(!q.empty()){
4     int u = q.front();
5     q.pop();
6     for(int v:adj[u]){
7         if (!vis[v]) {

```

```

8         vis[v] = 1;
9         q.push(v);
10    }
11 }
12 }

```

4.3 Topological Sort

```

1 vi ts;
2 queue<int> q;
3 for (int i = 1; i < n+1; i++)
4     if(!in[i])q.push(i);
5
6 while (!q.empty()) {
7     int u=q.front();
8     q.pop();
9     ts.pb(u);
10    for (int v : adj[u]) {
11        in[v]--;
12        if(!in[v])q.push(i);
13    }
14 }
15
16 if(sz(ts)!=n)cout <<"IMPOSSIBLE";

```

4.4 Flood Fill

```

1 int dx[] = {1,1,0,-1,-1,-1,0,1};
2 int dy[] = {0,1,1,1,0,-1,-1,-1};
3
4 int floodfill(int x,int y,char color1,char color2){
5     if(grid[x][y] == "outside")return 0;
6     if(grid[x][y] != color1)return 0;
7     int ans = 1;
8     grid[x][y] = color2;
9     for(int d=0;d<8;d++)ans += floodfill(x+dx[d],y+dy[d],color1,colr2);
10    return ans;
11 }

```

4.5 MST

4.5.1 Kruskal

```

1 struct Edge{

```

```

2   int u,v,w;
3   bool operator < (Edge b){return w<b.w;}
4 };
5
6 //inside main
7
8 vector<Edge> edges;
9 sort(all(edges));
10
11 DSU dsu(vertices);
12
13 for(int i=0;i<E;i++){
14     Edge front = edges[i];
15     if(!dsu.same(front.u,front.v)){
16         cost+= front.w;
17         dsu.merge(front.u,front.v);
18     }
19 }

```

4.5.2 Prims

```

1 vector<bool> vis(n);
2 priority_queue<pii, vector<pii>, greater<pii>> q;
3
4 auto process = [&](int u){
5     vis[u] = 1;
6     for(int i=0;i<sz(adj[u]);i++){
7         pii v = adj[u][i];
8         swap(v.S,v.F);
9         q.push(v);
10    }
11 }
12 process(0);
13
14 while(!q.empty()){
15     auto [w,u] = q.top();q.pop();
16     if(!vis[u]){cost += w;process(u);}
17 }

```

4.6 Dijkstra

4.6.1 Implementation

```

1 vll dist(n,INF64);
2 vector<bool> vis(n,0);
3 priority_queue<p11,vector<p11>,greater<p11>> pq;
4 pq.emplace(0,0);
5 dist[0] = 0;
6 vll cnt(n,0);//number of shortest paths
7 while(!pq.empty()){
8     auto [d1,u] = pq.top();
9     pq.pop();
10    if(vis[u])continue;
11    vis[u] = 1;
12    for(auto [v,c]:adj[u]){
13        if(dist[u]+c < dist[v]){
14            dist[v] = dist[u]+c;
15            pq.emplace(dist[v],v);
16            cnt[v] = cnt[u];
17        }else if(dist[u]+c == dist[v]){cnt[v]+=cnt[u]}%MOD;
18    }
19 }

```

4.6.2 Space-State graph

```

1 vector<vll> dist(n,vll(1001,INF64));
2 vector<vector<bool>> vis(n,vector<bool> (1001,0));
3 typedef array<ll,3> T;
4 priority_queue<T ,vector<T>,greater<T>> pq;
5 pq.push({0,0,state[0]});
6 dist[0][state[0]] = 0;
7 while(!pq.empty()){
8     auto [d,u,s] = pq.top();
9     pq.pop();
10    if(vis[u][s])continue;
11    vis[u][s] = 1;
12    for(auto [v,w]:adj[u]){
13        ll c = min(s,slow[v]);
14        //Nuevo estad cuando llegue a V
15        if(dist[u][s]+w*s<dist[v][c]){
16            dist[v][c] = dist[u][s]+w*s;
17            pq.push({dist[v][c],v,c});
18        }
19    }
20 }
```

4.7 Bellman-Ford

4.7.1 Path

```

1 vector<int> d(n, INF);
2 d[v] = 0;
3 vector<int> p(n, -1);
4
5 for (;;) {
6     bool any = false;
7     for (Edge e : edges)
8         if (d[e.a] < INF)
9             if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = d[e.a] + e.cost;
11                 p[e.b] = e.a;
12                 any = true;
13             }
14     if (!any)
15         break;
16 }
17
```

```

18 if (d[t] == INF)
19     cout << "No_path_from_" << v << "_to_" << t << ".";
20 else {
21     vector<int> path;
22     for (int cur = t; cur != -1; cur = p[cur])path.push_back(cur);
23     reverse(path.begin(), path.end());
24 }
```

4.7.2 Cycle

```

1 vector<int> d(n, INF);
2 d[v] = 0;
3 vector<int> p(n, -1);
4 int x;
5 for (int i = 0; i < n; ++i) {
6     x = -1;
7     for (Edge e : edges)
8         if (d[e.a] < INF)
9             if (d[e.b] > d[e.a] + e.cost) {
10                 d[e.b] = max(-INF, d[e.a] + e.cost);
11                 p[e.b] = e.a;
12                 x = e.b;
13             }
14 }
15 if (x == -1)cout << "No_negative_cycle_from_" << v;
16 else{
17     int y = x;
18     for (int i = 0; i < n; ++i)y = p[y];
19     vector<int> path;
20     for (int cur = y;; cur = p[cur]) {
21         path.push_back(cur);
22         if (cur == y && path.size() > 1)break;
23     }
24     reverse(path.begin(), path.end());
25 }
```

4.8 Floyd-Warshall

```

1 for (int i = 0; i < n; ++i)d[i][i] = 0;
2
3 for (int k = 0; k < n; ++k)
4     for (int i = 0; i < n; ++i)
5         for (int j = 0; j < n; ++j)
6             if (d[i][k] < INF && d[k][j] < INF)
```

```
7 | d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

4.9 SCC Kosaraju

```
1 | class SCC{
2 | private:
3 |     int n;
4 |     vector<vi> adj;
5 |     vector<vi> adj_t;
6 |     vector<vi> adj_cond;
7 |     vi order;
8 |     vector<bool> vis;
9 |     vi who;
10 |    int comp;
11 | public:
12 |    SCC(int n,vector<vi> &a,vector<vi>&b):n(n),adj(a),adj_t(b){
13 |        comp = -1;
14 |        who.resize(n);
15 |        vis.resize(n);
16 |        for(int i=0;i<n;i++)
17 |            if(!vis[i])dfs1(i);
18 |
19 |        vis.assign(n,0);
20 |        for(int i=n-1;i>=0;i--)
21 |            if(!vis[order[i]]){
22 |                comp++;
23 |                dfs2(order[i]);
24 |            }
25 |        adj_cond.resize(comp+1);
26 |        for(int u=0;u<n;u++)
27 |            for(auto v:adj[u])
28 |                if(who[v]!=who[u])
29 |                    adj_cond[who[u]].pb(who[v]);
30 |    }
31 |
32 |    void dfs1(int u) {
33 |        vis[u] = true;
34 |        for (int v:adj[u])
35 |            if (!vis[v])dfs1(v);
36 |        order.pb(u);
37 |    }
38 |
39 |    void dfs2(int u) {
```

```
40 |        vis[u] = 1;
41 |        for (int v : adj_t[u])
42 |            if (!vis[v])dfs2(v);
43 |        who[u] = comp;
44 |    }
45 |};
```

4.10 Flows

4.10.1 Dinic

```
1 | struct flowEdge{
2 |     int u,v;
3 |     ll cap,flow = 0;
4 |     flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
5 | };
6 |
7 | struct Dinic{
8 |     vector<flowEdge> edges;
9 |     vector<vi> adj;
10 |    int n,s,t;
11 |    int id = 0;
12 |    vi level, next;
13 |
14 |    queue<int> q;
15 |
16 |    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
17 |        adj.resize(n);
18 |        level.resize(n);
19 |        next.resize(n);
20 |        fill(all(level),-1);
21 |        level[s] = 0;
22 |        q.push(s);
23 |    }
24 |
25 |    void add(int u, int v, ll cap){
26 |        edges.emplace_back(u,v,cap);
27 |        edges.emplace_back(v,u,0);
28 |        adj[u].pb(id++);
29 |        adj[v].pb(id++);
30 |    }
31 |
32 |    bool bfs(){
33 |        while (!q.empty()){
```

```

34     int curr = q.front();
35     q.pop();
36     for (auto e : adj[curr]){
37         if (edges[e].cap - edges[e].flow < 1) continue;
38         if (level[edges[e].v] != -1) continue;
39         level[edges[e].v] = level[edges[e].u] + 1;
40         q.push(edges[e].v);
41     }
42 }
43 return level[t] != -1;
44 }
45
46 ll dfs(int u, ll flow){
47     if (flow == 0) return 0;
48     if (u == t) return flow;
49     for (auto& cid = next[u]; cid < adj[u].size(); cid++){
50         int e = adj[u][cid];
51         int v = edges[e].v;
52
53         if (level[edges[e].u] + 1 != level[v] || edges[e].cap - edges[e].
            flow < 1) continue;
54         ll f = dfs(v, min(flow, edges[e].cap - edges[e].flow));
55         if (f == 0) continue;
56         edges[e].flow += f;
57         edges[e ^ 1].flow -= f;
58         return f;
59     }
60     return 0;
61 }
62
63 ll maxFlow(){
64     ll flow = 0;
65     while (bfs()){
66         fill(all(next), 0);
67         for (ll f = dfs(s, INF64); f != 0; f = dfs(s, INF64)) flow += f;
68         fill(all(level), -1);
69         level[s] = 0;
70         q.push(s);
71     }
72     return flow;
73 }
74
75

```

```

76 vector<pii> minCut(){
77     vector<pii> ans;
78     fill(all(level), -1);
79     level[s] = 0;
80     q.push(s);
81     while (!q.empty()){
82         int curr = q.front();
83         q.pop();
84         for (int id = 0; id < adj[curr].size(); id++){
85             int e = adj[curr][id];
86             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow > 0){
87                 q.push(edges[e].v);
88                 level[edges[e].v] = level[edges[e].u] + 1;
89             }
90         }
91     }
92
93     for (int i = 0; i < level.size(); i++){
94         if (level[i] != -1){
95             for (int id = 0; id < adj[i].size(); id++){
96                 int e = adj[i][id];
97                 if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow == 0)
98                     ans.emplace_back(edges[e].u, edges[e].v);
99             }
100         }
101     }
102     return ans;
103 }
104
105 vector<pii> MaximumMatching(){
106     vector<pii> ans;
107     fill(all(level), -1);
108     level[s] = 0;
109     q.push(s);
110     while (!q.empty()){
111         int curr = q.front();
112         q.pop();
113         for (int id = 0; id < adj[curr].size(); id++){
114             int e = adj[curr][id];
115             if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow == 0 &&
                edges[e].flow != 0){
116                 q.push(edges[e].v);
117                 level[edges[e].v] = level[edges[e].u] + 1;

```

```

118     }
119 }
120 }
121 for (int i = 0; i < level.size(); i++){
122     if (level[i] != - 1)
123         for (int id = 0; id < adj[i].size(); id++){
124             int e = adj[i][id];
125             if (edges[e].u != s && edges[e].v != t && edges[e].cap - edges[e]
126                 ].flow == 0 && edges[e].flow != 011)
127                 ans.emplace_back(edges[e].u,edges[e].v);
128         }
129 }
130 return ans;
131 }
132 };

```

4.11 2-SAT

```

1 class SAT{
2 private:
3     int n;
4     vector<vi> adj;
5     vector<vi> adj_t;
6     vector<vi> adj_cond;
7     vi order;
8     vector<bool> vis;
9     vi who;
10    vector<bool> ans;
11    int comp;
12 public:
13
14    SAT(int m=0){
15        n = (m<<1);
16        adj.resize(n);
17        adj_t.resize(n);
18        vis.assign(n,0);
19        who.assign(n,-1);
20    }
21
22    void dfs1(int u) {
23        vis[u] = true;
24        for (int v:adj[u])

```

```

25        if (!vis[v])dfs1(v);
26        order.pb(u);
27    }
28
29    void dfs2(int u) {
30        vis[u] = 1;
31        for (int v : adj_t[u])
32            if (!vis[v])dfs2(v);
33        who[u] = comp;
34    }
35
36    //~a->b ~b->a
37    void addOR(int a, bool af, int b, bool bf) {
38        a += a + (af ^ 1);
39        b += b + (bf ^ 1);
40        adj[a ^ 1].push_back(b);
41        adj[b ^ 1].push_back(a);
42        adj_t[b].push_back(a ^ 1);
43        adj_t[a].push_back(b ^ 1);
44    }
45
46    //( a OR b ) ^ (~a or ~b)
47    void addXOR(int a, bool af, int b, bool bf) {
48        addOR(a, af, b, bf);
49        addOR(a, !af, b, !bf);
50    }
51    //(a -> b)
52    void _add(int a,bool af,int b,bool bf) {
53        a += a + (af ^ 1);
54        b += b + (bf ^ 1);
55        adj[a].push_back(b);
56        adj_t[b].push_back(a);
57    }
58
59    //(a->b)^(b->a)
60    void add(int a,bool af,int b,bool bf) {
61        _add(a, af, b, bf);
62        _add(b, !bf, a, !af);
63    }
64
65    bool solve_SAT(){
66        for(int i=0;i<n;i++)
67            if(!vis[i])dfs1(i);

```

```

68
69     comp = -1;
70     vis.assign(n,0);
71     for(int i=sz(order)-1;i>=0;i--)
72         if(!vis[order[i]]){
73             comp++;
74             dfs2(order[i]);
75         }
76     ans.assign(n/2,0);
77     for(int i=0;i<n;i+=2){
78         if(who[i] == who[i+1])return 0;
79         ans[i/2] = (who[i] > who[i+1]);
80     }
81     return 1;
82 }
83
84 bool operator[] (int i){
85     return ans[i];
86 }
87
88 };

```

4.12 Euler Paths and cycles

4.12.1 Directed Graphs

```

1 struct EulerSolver{
2 private:
3     vector<vector<pii>> adj;
4     int n,edges;
5     vi in,out;
6     vi ans,edge_ans;
7     int root;
8     vi done;
9     void dfs(int u){
10         while(done[u] < sz(adj[u])){
11             auto [v,id] = adj[u][done[u]++];
12             dfs(v);
13             edge_ans.pb(id);
14         }
15         ans.pb(u);
16     }
17 public:
18     EulerSolver(vector<vector<pii>> &a,int t):adj(a){

```

```

19     n = t;
20     edges = 0;
21     in.assign(n,0);
22     out.assign(n,0);
23     done.assign(n,0);
24     root = -1;
25     for(int u=0;u<n;u++){
26         for(auto v:adj[u]){
27             edges++;
28             in[v.F]++;
29             out[u]++;
30         }
31     }
32
33
34     bool possible(){
35         int cnt1 =0,cnt2 =0;
36         bool ok = 1;
37         for(int i=0;i<n;i++){
38             if (in[i] - out[i] == 1) cnt1++;
39             if (out[i] - in[i] == 1) cnt2++, root = i;
40             if (abs(in[i] - out[i]) > 1) ok = 0;
41         }
42         if(cnt1 > 1 || cnt2 > 1)ok = 0;
43         if(!ok)return 0;
44
45         if(root == -1)
46             for(int i=0;i<n;i++)
47                 if(out[i])root = i;
48
49         if(root == -1)return 1;
50         dfs(root);
51
52         if(sz(ans) != edges+1)return 0;
53         reverse(all(ans));
54         reverse(all(edge_ans));
55         return 1;
56     }
57
58     int size(){
59         return ans.size();
60     }
61

```



```

62 int operator()(int i,bool x){
63     if(x)return ans[i];
64     else return edge_ans[i];
65 }
66 };

```

4.12.2 Unirected Graphs

```

1 struct EulerSolver{
2 private:
3     vector<vector<pii>> adj;
4     int n,edges;
5     vi ans,edge_ans;
6     vi deg;
7     int root;
8     vi done;
9     vector<bool> vis;
10
11 void dfs(int u){
12     while(done[u] < sz(adj[u])){
13         auto [v,id] = adj[u][done[u]++];
14         if(vis[id])continue;
15         vis[id]=1;
16         dfs(v);
17         edge_ans.pb(id);
18     }
19     ans.pb(u);
20 }
21 public:
22 EulerSolver(vector<vector<pii>> &a,int t):adj(a),n(t){
23     edges = 0;
24     done.assign(n,0);
25     deg.assign(n,0);
26     root = -1;
27     for(int u=0;u<n;u++){
28         for(auto v:adj[u]){
29             edges++;
30             deg[v.F]++;
31             deg[u]++;
32         }
33     vis.assign(edges/2,0);
34 }
35

```

```

36 bool possible(){
37     int odd = 0;
38     for(int i=0;i<n;i++){
39         if((deg[i]/2)&1){
40             odd++;
41             root = i;
42         }
43
44
45     if(odd>2)return 0;
46
47     if(root == -1)
48         for(int i=0;i<n;i++){
49             if(deg[i]){root = i;break;}
50
51     if(root == -1)return 1;
52     dfs(root);
53
54     if(sz(ans) != edges/2+1)return 0;
55     reverse(all(ans));
56     reverse(all(edge_ans));
57     return 1;
58 }
59
60 int size(){
61     return ans.size();
62 }
63
64 int operator()(int i,bool x){
65     if(x)return ans[i];
66     else return edge_ans[i];
67 }
68 };

```

5 Trees

5.1 LCA - Binary Lifting

5.1.1 Normal LCA

```

1  const int MAX = 1e5+5, LG=18;
2  vi deep(MAX); // Si tiene peso vi cost(MAX)
3  int par[MAX][LG+1];
4
5  void dfs(int u = 1, int p=0) { // U = raiz del arbol
6      par[u][0] = p;
7      deep[u] = deep[p]+1; // cost[u] += cost[p]
8      for(int i=1; i<=LG; i++) par[u][i] = par[par[u][i-1]][i-1];
9      for(int v: adj[u]) {
10         if(v!=p) {
11             dfs(v, u);
12         }
13     }
14 }
15
16 int lca(int u, int v) {
17     if(deep[u] < deep[v]) swap(u, v);
18     for(int k=LG; k>=0; k--)
19         if(deep[par[u][k]] >= deep[v])
20             u = par[u][k];
21     if(u==v) return u;
22     for(int k=LG; k>=0; k--)
23         if(par[u][k] != par[v][k])
24             u=par[u][k], v=par[v][k];
25     return par[u][0];
26 }
27
28 int dist(int u, int v) {
29     int lc = lca(u, v);
30     return deep[u] + deep[v] - (deep[lc] << 1);
31 }
32
33 int kth(int u, int k) {
34     assert(k>=0);
35     for(int i=0; i<=LG; i++)
36         if(k && (1<<i)) u = par[u][i];
37     return u;

```

```

38 }
39
40 int isanc(int u, int g) {
41     int k = dep[u] - dep[g];
42     return k >= 0 && kth(u, k) == g;
43 }

```

5.1.2 Min/Max weight on the path

```

1  const int MAX = 1e5+5, LG=18;
2  vector<vector<pii>> adj(MAX, vector<pii> ());
3  vi deep(MAX);
4  int par[MAX][LG+1];
5  int mn[MAX][LG+1];
6  int mx[MAX][LG+1];
7
8  void dfs(int u=1, int p=0, int costmn = INF, int costmx = -INF) {
9      par[u][0] = p;
10     deep[u] = deep[p]+1;
11
12     if(p!=0) {
13         mn[u][0] = costmn;
14         mx[u][0] = costmx;
15     }
16     for(int i=1; i<=LG; i++) {
17         par[u][i] = par[par[u][i-1]][i-1];
18         mn[u][i] = min(mn[u][i-1], mn[par[u][i-1]][i-1]);
19         mx[u][i] = max(mx[u][i-1], mx[par[u][i-1]][i-1]);
20     }
21
22     for(auto [v, t]: adj[u]) {
23         if(v!=p) {
24             dfs(v, u, t, t);
25         }
26     }
27 }
28
29 pii minmax(int u, int v) {
30     pii ans = {INF, -INF};
31     if(deep[u] < deep[v]) swap(u, v);
32     for(int k=LG; k>=0; k--) if(deep[par[u][k]] >= deep[v]) {
33         ans.F = min(ans.F, mn[u][k]);
34         ans.S = max(ans.S, mx[u][k]);

```

```

35     u = par[u][k];
36 }
37 if(u==v)return ans;
38 for(int k=LG;k>=0;k--)if(par[u][k]!=par[v][k]){
39     ans.F = min({ans.F,mn[u][k],mn[v][k]});
40     ans.S = max({ans.S,mx[u][k],mx[v][k]});
41     u=par[u][k],v=par[v][k];
42 }
43 ans.F = min({ans.F,mn[u][0],mn[v][0]});
44 ans.S = max({ans.S,mx[u][0],mx[v][0]});
45
46 return ans;
47 }

```

5.1.3 Union path

```

1 pair<int, int> merge(pair<int, int> x, pair<int, int> y) {
2     if (x.first == 0)return y; //where {0,0} is a null path
3     if (y.first == 0)return x;
4     if (x.first == -1 || y.first == -1) return { -1, -1};
5     vector<int> can = {x.first, x.second, y.first, y.second};
6     int a = can[0];
7     for (int u : can)
8         if (dep[u] > dep[a])
9             a = u;
10    int b = -1;
11    for (int u : can)
12        if (!isanc(a, u)) {
13            if (b == -1) b = u;
14            if (dep[b] < dep[u]) b = u;
15        }
16    if (b == -1) {
17        b = can[0];
18        for (int u : can)
19            if (dep[u] < dep[b])
20                b = u;
21        return {a, b};
22    }
23    int g = lca(a, b);
24    for (int u : can) {
25        if (u == a || u == b)continue;
26        if (dep[u] < dep[g] || (!isanc(a, u) && !isanc(b, u)))return { -1,
-1};

```

```

27     }
28     return {a, b};
29 }

```

5.1.4 Intersection Path

```

1 pair<int, int> getCommonPath(int u, int a, int v, int b) {
2     // returns common path between u->a and v->b where u is ancestor a and v
    is ancestor b
3     if (!isanc(a, v)) return make_pair(0, 0);
4     int x = lca(a, b);
5     if (dep[v] < dep[u]) {
6         if (isanc(x, u))
7             return make_pair(u, x);
8     }else{
9         if (isanc(x, v))
10            return make_pair(v, x); // v->x is the common path
11    }
12    return make_pair(0, 0);
13 }
14 pair<int, int> path_union_light(pair<int, int> x, pair<int, int> y) {
15     if (x.first == 0) return y; //where {0,0} is a null path
16     if (y.first == 0) return x;
17     vector<int> can = {x.first, x.second, y.first, y.second};
18     int a = can[0];
19     for (int u : can) if (dep[u] > dep[a]) a = u;
20     int b = -1;
21     for (int u : can)if (!isanc(a, u)) {
22         if (b == -1) b = u;
23         if (dep[b] < dep[u]) b = u;
24     }
25     if (b == -1) {
26         b = can[0];
27         for (int u : can) if (dep[u] < dep[b]) b = u;
28         return {a, b};
29     }
30     return {a, b};
31 }
32 // returns the intersection of two paths (a,b) and (c,d)
33 // {0,0} for null path
34 pair<int, int> path_intersection(int a, int b, int c, int d) {
35     if (a == 0 || b == 0) return {0, 0};
36     int u = lca(a, b), v = lca(c, d); // splits both paths in two chains

```

```

    from lca to one node
37 pair<int, int> x, y;
38 x = getCommonPath(u, a, v, c);
39 y = getCommonPath(u, a, v, d);
40 pair<int, int> a1 = path_union_light(x, y);
41 x = getCommonPath(u, b, v, c);
42 y = getCommonPath(u, b, v, d);
43 pair<int, int> a2 = path_union_light(x, y);
44 return path_union_light(a1, a2);
45 }

```

5.2 Centroid Decomposition

```

1 class Centroid{
2 private:
3     vector<vi> tree;
4     vi par;
5     vi sub;
6     vector<bool> check;
7     int n;
8 public:
9     Centroid(vector<vi> &t):tree(t){
10         n = tree.size();
11         par.resize(n);
12         sub.resize(n);
13         check.resize(n);
14         build();
15     }
16
17     void build(int u = 0, int p = -1){
18         dfs(u, p);
19         int c = get_centroid(u,p,sub[u]);
20         check[c] = 1;
21         par[c] = p+1;
22
23         for (auto v : tree[c]){
24             if(!check[v])
25                 build(v, c);
26         }
27     }
28
29     int dfs(int u,int p){
30

```

```

31         if(check[u])return 0;
32         sub[u] = 1;
33         for(int v:tree[u])
34             if(v!=p)sub[u]+=dfs(v,u);
35
36         return sub[u];
37     }
38
39     int get_centroid(int u,int p,int x){
40         for(auto v:tree[u])
41             if(v!=p && sub[v]*2>x && !check[v])return get_centroid(v,u,x);
42
43         return u;
44     }
45
46     int operator[](int i){
47         return par[i];
48     }
49 };

```

5.3 Euler Tour Technique

```

1 int timer = 0;
2 vi start,final;
3 vector<vi> adj;
4 void dfs(int u = 0,int prev = -1){
5     start[u]=timer++;
6     for(int v:adj[u]){
7         if(v!=prev)dfs(v,u);
8     }
9     final[u] = timer;
10 }
11 // [start,final)

```

5.4 Tree Matching

```

1 vector<vll> dp(2,vll(MAX));
2
3 void dfs(ll u = 0, ll p=-1){
4     dp[0][u] = 0;
5     dp[1][u] = -INF;
6
7     for(ll v:adj[u]){
8         if(v == p)continue;

```

```

9     dfs(v,u);
10
11     dp[0][u] += max(dp[1][v], dp[0][v]);
12
13     ll opt = min(dp[0][v] - dp[1][v], 0ll);
14
15     dp[1][u] = max(dp[1][u], opt);
16 }
17
18 dp[1][u] += dp[0][u];
19 dp[1][u]++;
20 }

```

5.5 Diameter

```

1 vector<vector<pii>> adj;
2 vector<bool> vis;
3
4 int bfs(int n, vll &d, int v = 0){
5     vis.assign(n, 0);
6     d.assign(n, 0);
7     d[v] = 0;
8     queue<int> q;
9     q.push(v);
10    pll last = {v, 0};
11    vis[v] = 1;
12    while(!q.empty()){
13        int u = q.front(); q.pop();
14        for(auto [w, c]: adj[u]){
15            if(vis[w]) continue;
16            d[w] = d[u] + c;
17            q.push(w);
18            vis[w] = 1;
19            if(d[w] > last.S) last = {w, d[w]};
20        }
21    }
22    return int(last.F);
23 }
24
25 void solve(){
26     int n; cin >> n;
27     adj.assign(n, vector<pii> ());
28     for(int i=1; i<n; i++){

```

```

29     int a, b; ll c;
30     cin >> a >> b >> c;
31     adj[--a].pb({--b, c});
32     adj[b].pb({a, c});
33 }
34 vll dx(n), dy(n);
35 int a = bfs(n, dx);
36 int b = bfs(n, dy, a);
37 bfs(n, dx, b);
38
39 for(int i=0; i<n; i++){
40     cout << max(dx[i], dy[i]) << "\n" [i==(n-1)];
41 }
42
43 }

```

5.6 Heavy Light Decomposition

```

1 class HLD{
2 private:
3     int n;
4     vector<vi> adj;
5     vi parent;
6     vi heavy;
7     vi depth;
8     vi root;
9     vi treePos;
10    Segment_Tree tree;
11
12    int dfs(int u = 0){
13        int size = 1, mx_sub = 0;
14        for(auto v: adj[u]){
15            if(v != parent[u]){
16                parent[v] = u;
17                depth[v] = depth[u] + 1;
18                int sub = dfs(v);
19                if(sub > mx_sub){
20                    heavy[u] = v;
21                    mx_sub = sub;
22                }
23                size += sub;
24            }
25            return size;

```

```

26 }
27
28 template <class BinaryOperation>
29 void processPath(int u, int v, BinaryOperation op) {
30     for (; root[u] != root[v]; v = parent[root[v]]) {
31         if (depth[root[u]] > depth[root[v]]) swap(u, v);
32         op(treePos[root[v]], treePos[v] + 1);
33     }
34     if (depth[u] > depth[v]) swap(u, v);
35     op(treePos[u], treePos[v] + 1);
36 }
37
38 void init(vi &a){
39     for(int i=0,crt=0;i<n;++i){
40         if(parent[i] == -1 || heavy[parent[i]]!=i){
41             for(int j = i;j!=-1;j = heavy[j]){
42                 root[j] = i;
43                 treePos[j] = crt++;
44             }
45         }
46     }
47     tree = Segment_Tree(n);
48     for(int i=0;i<n;i++){
49         tree.update(treePos[i],a[i]);
50     }
51 }
52
53 public:
54
55 HLD(int n,vector<vi> &vale,vi &a):n(n),adj(vale){
56     parent.assign(n,0);
57     heavy.assign(n,-1);
58     depth.assign(n,0);
59     root.resize(n);
60     treePos.assign(n,0);
61     parent[0] = -1;
62     dfs();
63     init(a);
64 }
65
66 void update(int u,int &val){
67     tree.update(treePos[u],val);
68 }

```

```

69
70 void updatePath(int u, int v, int value) {
71     processPath(u, v, [this, &value](int l, int r) {
72         tree.update_range(l, r, value);
73     });
74 }
75
76 int query(int u,int v){
77     int ans = 0;
78     processPath(u,v,[this,&ans](int l,int r){
79         ans = max(ans,tree.query(l,r));
80     });
81     return ans;
82 }
83 };

```

6 Math

6.1 FFT

```

1 struct base {
2     double a, b;
3     base(double a = 0, double b = 0) : a(a), b(b) {}
4     const base operator + (const base &c) const{
5         return base(a + c.a, b + c.b);
6     }
7     const base operator - (const base &c) const{
8         return base(a - c.a, b - c.b);
9     }
10    const base operator * (const base &c) const{
11        return base(a * c.a - b * c.b, a * c.b + b * c.a);
12    }
13 };
14 void fft(vector<base> &p, bool inv = 0) {
15     int n = p.size(), i = 0;
16     for(int j = 1; j < n - 1; ++j) {
17         for(int k = n >> 1; k > (i ^= k); k >>= 1);
18         if(j < i) swap(p[i], p[j]);
19     }
20
21     for(int l = 1, m; (m = l << 1) <= n; l <=<= 1) {
22         double ang = 2 * PI / m;
23         base wn = base(cos(ang), (inv ? 1. : -1.) * sin(ang)), w;

```

```

24   for(int i = 0, j, k; i < n; i += m) {
25       for(w = base(1, 0), j = i, k = i + 1; j < k; ++j, w = w * wn) {
26           base t = w * p[j + 1];
27           p[j + 1] = p[j] - t;
28           p[j] = p[j] + t;
29       }
30   }
31 }
32
33 if(inv) for(int i = 0; i < n; ++i) p[i].a /= n, p[i].b /= n;
34 }
35 vector<ll> multiply(vi &a, vi &b) {
36     int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
37     while(sz < t) sz <= 1;
38     vector<base> x(sz), y(sz), z(sz);
39     for(int i = 0; i < sz; ++i) {
40         x[i] = i < (int)a.size() ? base(a[i], 0) : base(0, 0);
41         y[i] = i < (int)b.size() ? base(b[i], 0) : base(0, 0);
42     }
43     fft(x), fft(y);
44     for(int i = 0; i < sz; ++i) z[i] = x[i] * y[i];
45     fft(z, 1);
46     vector<ll> ret(sz);
47     for(int i = 0; i < sz; ++i) ret[i] = (long long) round(z[i].a);
48     while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
49     return ret;
50 }

```

6.1.1 XOR

$$a + b = a \oplus b + 2(a \& b)$$

$$a + b = a|b + a \& b$$

$$a \oplus b = a|b - a \& b$$

$$1 \oplus 2 \oplus 3 \oplus \dots \oplus (4k - 1) = 0 \quad k \geq 0$$

6.2 Theorems

7 Strings

7.1 Suffix Array

```

1 struct suffix{
2     int index;
3     pii rank;
4     bool operator<(suffix b){return rank<b.rank;}
5     bool operator>(suffix b){return rank>b.rank;}
6 };
7
8 class Suffix_Array{
9     public:
10    string s;int n;
11    vector<suffix> suffixes;
12    vi lcp;
13    Suffix_Array(string x){
14        s = x+"$";
15        n = sz(s);
16        suffixes.resize(n);
17    }
18
19
20    void radix_sort(vector<suffix> &suff){
21        for(int i: vi{2,1}){
22            auto key = [&](const suffix &x){
23                return i == 1?x.rank.F:x.rank.S;
24            };
25
26            int mx = 0;
27            for (const auto &i:suff) { mx = max(mx, key(i)); }
28            vector<int> occs(mx + 1);
29            for (const auto &i:suff)occs[key(i)]++;
30            vector<int> start(mx + 1);
31            for (int i=1;i<=mx; i++) start[i] = start[i-1]+occs[i-1];
32
33            vector<suffix> new_arr(suff.size());
34            for (const auto &i : suff) {
35                new_arr[start[key(i)]] = i;
36                start[key(i)]++;
37            }
38            suff = new_arr;
39        }
40    }
41
42    void build(){

```

```

43 for(int i=0;i<n;i++)suffixes[i].index = i , suffixes[i].rank = {s[i
    ],(i+1<n?s[i+1]:-1)};
44 sort(all(suffixes));
45 vi equiv(n);
46
47 for(int i=1;i<n;i++){
48     auto c_val = suffixes[i].rank;
49     int cs = suffixes[i].index;
50     auto p_val = suffixes[i - 1].rank;
51     int ps = suffixes[i-1].index;
52     equiv[cs] = equiv[ps] + (c_val > p_val);
53 }
54
55 for(int cmp_ant = 1;cmp_ant <n; cmp_ant<=<=1){
56     for(auto &x:suffixes)
57         x.rank = {equiv[x.index],equiv[(x.index+cmp_ant)%n]};
58
59     //Std sort O(nlognlogn) Radix Sort O(nlogn)
60     radix_sort(suffixes);
61
62     for(int i=1;i<n;i++){
63         auto c_val = suffixes[i].rank;
64         int cs = suffixes[i].index;
65         auto p_val = suffixes[i - 1].rank;
66         int ps = suffixes[i-1].index;
67         equiv[cs] = equiv[ps] + (c_val > p_val);
68     }
69 }
70 }
71
72 void build_lcp(){
73     vector<int> suff_ind(n);
74     for (int i = 0; i < n; i++)suff_ind[suffixes[i].index] = i;
75     lcp.resize(n-1);
76     int start_at = 0;
77     for (int i = 0; i < n - 1; i++) {
78         int prev = suffixes[suff_ind[i] - 1].index;
79         int curr_cmp = start_at;
80         while (s[i + curr_cmp] == s[prev + curr_cmp])curr_cmp++;
81         lcp[suff_ind[i] - 1] = curr_cmp;
82         start_at = max(curr_cmp - 1, 0);
83     }
84 }

```

```

85
86 void show(){
87     for(int i=0;i<n;i++)cout << suffixes[i].index << "\n"[(i+1)==n];
88 }
89 };

```

8 Others

8.1 Binpow

```

1 ll binpow(ll a,ll b){
2     a%=MOD;
3     ll ans = 1;
4     while(b){
5         if(b&1)(ans*=a)%=MOD;
6         (a*=a)%=MOD;
7         b>>=1;
8     }
9     return ans;
10 }

```

8.2 Matrix exponentiation

```

1 struct Mat {
2     int n, m;
3     vector<vector<int>>> a;
4     Mat() { }
5     Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
6     Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0;
7         a = v; }
8     inline void make_unit() {
9         assert(n == m);
10        for (int i = 0; i < n; i++) {
11            for (int j = 0; j < n; j++) a[i][j] = i == j;
12        }
13    }
14    inline Mat operator + (const Mat &b) {
15        assert(n == b.n && m == b.m);
16        Mat ans = Mat(n, m);
17        for(int i = 0; i < n; i++) {
18            for(int j = 0; j < m; j++) {
19                ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
20            }
21        }
22    }

```



```

20 }
21 return ans;
22 }
23 inline Mat operator - (const Mat &b) {
24     assert(n == b.n && m == b.m);
25     Mat ans = Mat(n, m);
26     for(int i = 0; i < n; i++) {
27         for(int j = 0; j < m; j++) {
28             ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
29         }
30     }
31     return ans;
32 }
33 inline Mat operator * (const Mat &b) {
34     assert(m == b.n);
35     Mat ans = Mat(n, b.m);
36     for(int i = 0; i < n; i++) {
37         for(int j = 0; j < b.m; j++) {
38             for(int k = 0; k < m; k++) {
39                 ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] % mod)
40                     % mod;
41             }
42         }
43     }
44     return ans;
45 }
46 inline Mat pow(long long k) {
47     assert(n == m);
48     Mat ans(n, n), t = a; ans.make_unit();
49     while (k) {
50         if (k & 1) ans = ans * t;
51         t = t * t;
52         k >>= 1;
53     }
54     return ans;
55 }
56 inline Mat& operator += (const Mat& b) { return *this = (*this) + b; }
57 inline Mat& operator -= (const Mat& b) { return *this = (*this) - b; }
58 inline Mat& operator *= (const Mat& b) { return *this = (*this) * b; }
59 inline bool operator == (const Mat& b) { return a == b.a; }
60 inline bool operator != (const Mat& b) { return a != b.a; }
61 };

```

```

62 // Usage
63 // Mat a(n, n);
64 // Mat b(n, n);
65 // Mat c = a * b;
66 // Mat d = a + b;
67 // Mat e = a - b;
68 // Mat f = a.pow(k);
69 // a.a[i][j] = x;

```

8.3 Random

```

1 | mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());

```